
Spring Boot + Kafka Guide

Event-Driven Microservices

1. Introduction

Apache Kafka is a distributed messaging system used for communication between microservices.

Flow:

Producer → Kafka Topic → Consumer

Kafka enables asynchronous communication between services.

2. Why Kafka?

Traditional System

Service A → calls → Service B

Problems:

- Tight coupling
 - Slow response
 - Failure dependency
-

Kafka Solution

Service A → sends event → Kafka → Service B consumes later

Benefits:

- Loose coupling
 - High scalability
 - Fault tolerance
-

3. Prerequisites

- Java 17+
 - Spring Boot project
 - Spring Web dependency
 - Spring Kafka dependency
 - Docker installed
-

4. Kafka Docker Setup (YAML Explanation)

We use Docker Compose to run Kafka easily without manual installation.

What is YAML here?

YAML (Yet Another Markup Language) is used to define container configuration in a readable format.

It tells Docker:

- which services to run
 - how to configure Kafka
 - which ports to expose
 - how Kafka behaves internally
-

docker-compose.yml

```
services:
  kafka:
    image: apache/kafka:latest
    container_name: kafka
    ports:
      - "9092:9092"
```

Explanation:

- `services` → defines containers
 - `kafka` → service name
 - `image` → Kafka Docker image
 - `container_name` → name of running container
 - `ports` → exposes Kafka on port 9092
-

Kafka Configuration

```
environment:  
  KAFKA_NODE_ID: 1  
  KAFKA_PROCESS_ROLES: broker,controller  
  KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
```

Explanation:

- KAFKA_NODE_ID → unique ID of Kafka node
 - PROCESS_ROLES → Kafka runs as broker + controller
 - LISTENERS → defines internal communication ports
-

Kafka Networking Setup

```
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092  
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER  
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Explanation:

- ADVERTISED_LISTENERS → how clients connect to Kafka
 - CONTROLLER_LISTENER_NAMES → internal coordination
 - REPLICATION_FACTOR → number of replicas (1 for local setup)
-

5. Run Kafka

Command:

```
docker compose up -d
```

6. application.properties

```
spring.kafka.bootstrap-servers=localhost:9092  
spring.kafka.consumer.group-id=my-group  
spring.kafka.consumer.auto-offset-reset=earliest
```

Explanation:

- bootstrap-servers → Kafka server address
 - group-id → consumer group name
 - auto-offset-reset → starts reading from earliest message if no offset exists
-

7. Producer Service

```
@Service
public class KafkaProducerService {

    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    public void send(String msg) {
        kafkaTemplate.send("demo-topic", msg);
    }
}
```

Explanation:

- Sends messages to Kafka topic
 - Uses KafkaTemplate (Spring abstraction)
 - Works asynchronously
-

8. REST Controller

```
@RestController
@RequestMapping("/kafka")
public class KafkaController {

    @Autowired
    KafkaProducerService producer;

    @GetMapping("/send/{msg}")
    public String send(@PathVariable String msg) {
        producer.send(msg);
        return "Sent";
    }
}
```

Explanation:

- Exposes REST API
 - Triggers Kafka producer
 - Entry point for message flow
-

9. Consumer Service

```
@Service
public class KafkaConsumer {

    @KafkaListener(topics = "demo-topic", groupId = "my-group")
    public void listen(String msg) {
        System.out.println(msg);
    }
}
```

Explanation:

- Listens to Kafka topic
 - Automatically consumes messages
 - Part of consumer group
-

10. Message Flow

Browser → Controller → Producer → Kafka → Consumer → Console

11. Summary

- Producer sends messages to Kafka
 - Consumer reads messages from Kafka
 - Kafka enables asynchronous communication
 - Topics store messages
 - Consumer groups enable scalability
-